

Writeup CTF – Maniac

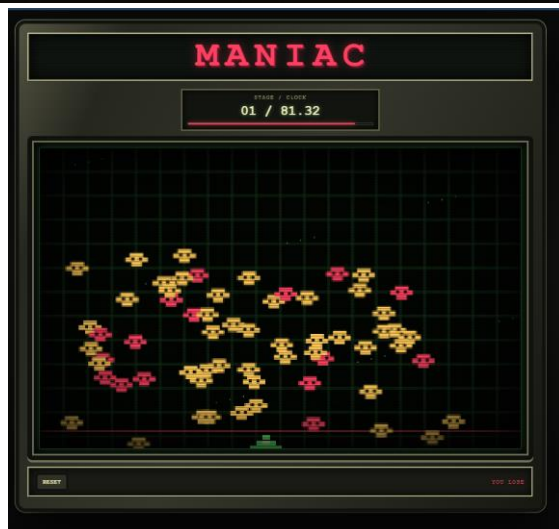
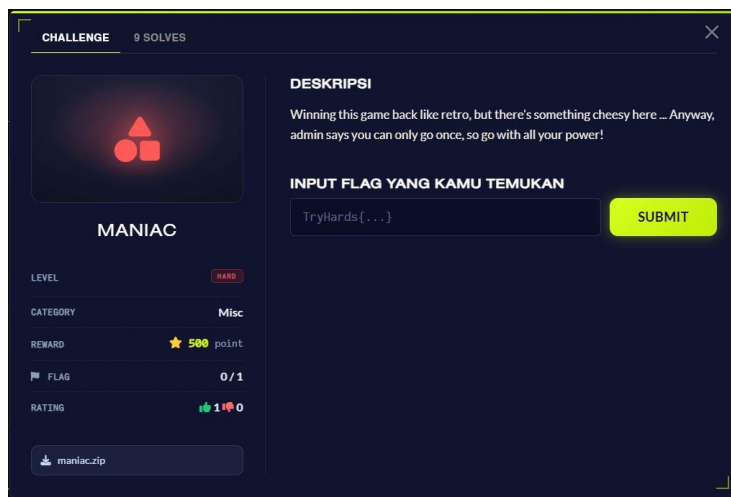
Solved by 0xJC

Kategori: Misc / Difficulty: Hard

Flag: TryHards{m4n!ac_sh0o0o0o0o0tzz}

file challenge ini ada dua biji file: index.html sama maniac.wasm dan ada dua clue:

- "there's something cheesy here"
- "admin says you can only go once, so go with all your power!"



1. Recon Awal - Baca index.html

Sebelum megang WASM-nya, saya baca dulu index.html buat memahami gimana game ini seharusnya dimainkan dari sisi browser. Ada beberapa hal yang langsung saya catat:

1. WASM-nya di-load tanpa import function apapun:
WebAssembly.instantiate(wasm, {}). Artinya seluruh logic game itu self-contained di dalam WASM, gak butuh helper apapun dari JS.
2. Seed dan durasi game di-hardcode: $F = 0x13579bdf$, $T = 90000$ (90 detik).
Ini penting banget artinya game ini fully deterministic, hasil sama persis tiap kali di-init dengan nilai yang sama.
3. Ada object mapping field di JS:

```
const o = {x:0,y:1,am:2,cl:3,sc:4,al:5,ph:6,tm:7,ec:8,ro:9,dy:10,
           au:11,ms:12,cd:13,sd:14,es:15,ea:16,tk:17,lv:18};
```

Tapi pas saya cek lagi, ternyata JS aslinya cuma make o.lv, o.tm, o.ph, sama o.al buat nampilin UI. Field lain kayak am, cl, sc, ec, ro, dy, au, ms, cd, sd, es, ea, tk sama sekali gak dipake di JS. Jadi nama-nama itu bisa aja cuma label yang menyesatkan, belum tentu sama dengan fungsi aslinya di WASM. Ini saya simpen sebagai catatan buat nanti divalidasi ulang.

Export function dari WASM cuma ada 6, saya cek langsung lewat Node.

2. Nyoba Load WASM Langsung di Node.js

Karena WASM-nya gak butuh import apapun, saya mikir daripada trace lewat DevTools browser yang lambat, mending saya jalanin & eksperimen langsung dari Node.js. Jauh lebih cepat buat iterasi coba-coba.

```
// step1_load.js
const fs = require('fs');

WebAssembly.instantiate(fs.readFileSync('maniac.wasm'), {}).then(a => {
  const x = a.instance.exports;
  console.log('Exports:', Object.keys(x));
});
```

```
EVIDENCE
> node step1_load.js
Exports: [ 'm', 'i', 't', 's', 'v', 'l' ]
PS C:\Users\LENOVO\Downloads\maniac\evidence>
```

Ketemu 6 export: m (memory) dan ini exported, artinya saya bisa baca/tulis langsung dari JS!), i (init), t (tick), s, v (cek menang), l (panjang flag). Dari nama & cara maknanya di JS, pola besarnya udah kebayang: main game → kalau menang, v() balikin pointer ke flag, l() balikin panjangnya.

3. Init Game & Lihat State Awal

Selanjutnya saya init game-nya pake seed & durasi yang sama kayak di index.html, terus dump seluruh state array-nya (disimpen sebagai int32 array mulai address 0 di memory WASM).

```
// step2_init.js
const x = ...;
x.i(0x13579bdf, 90000, 0); // seed, durasi, reroll
const state = new Int32Array(x.m.buffer);
console.log(Array.from(state.slice(0, 22)));
```

```

EVIDENCE
> node step2_init.js
State awal (index 0-21):
[
  450000,    900000,      96,
    0,        0,        1,
    0,    90000,        0,
    0,        0, -1059140866,
    0,        0,  324508639,
  8414013, -475658378, 2118544133,
    1,        0,        0,
    54921
]
PS C:\Users\LENOVO\Downloads\maniac\evidence>

```

State awal game 22 field int32 pertama, index 21 kebetulan bernilai 54921

Catatan kecil yang saya simpen: index 21 nilainya 54921. Belum tau relevansinya apa, tapi angka yang aneh & spesifik kayak gini biasanya bukan kebetulan.

4. Coba Main Manual - Ngerti Mekanik Dasar

Sebelum baca bytecode, saya coba main dulu (lewat kode, bukan klik manual) biar punya feeling soal mekanik game-nya. Saya panggil `t(move, dt)` berulang - `move` itu bitmask (`bit0=kiri`, `bit1=kanan`, `bit2=tembak`), `dt` itu delta waktu per frame.

```

// step3_play.js - terus nembak (move=4), gak gerak
for (let f = 0; f < 600; f++) {
  x.t(4, 16);
  const s = z();
  if (f % 100 === 0) console.log(f, s[o.sc], s[o.am], s[o.al], s[o.ph]);
  if (s[o.ph] !== 0) { console.log('GAME OVER'); break; }
}

```

```

EVIDENCE
> node step3_play.js
frame 0 | score= 0 | ammo= 95 | alive= 1 | phase= 0
frame 100 | score= 549 | ammo= 75 | alive= 1 | phase= 0
frame 200 | score= 1185 | ammo= 55 | alive= 1 | phase= 0
frame 300 | score= 1578 | ammo= 35 | alive= 1 | phase= 0
frame 400 | score= 1775 | ammo= 15 | alive= 1 | phase= 0
frame 500 | score= 1775 | ammo= 0 | alive= 1 | phase= 0
>>> GAME OVER di frame 543 phase= 1
PS C:\Users\LENOVO\Downloads\maniac\evideance>

```

Score naik, tapi ammo (idx2) abis dari 96 ke 0 dengan cepat dan gak regen; lalu player mati di frame 543

Insight pertama: ammo cuma 96 dan gak regen sama sekali. Berarti kill maksimal yang mungkin dalam satu game itu dibatasiin ketat, paling banter seratusan.

Terus saya coba pendekatan lebih agresif: abis mati, saya paksa balikin status 'hidup' & 'masih main' langsung dari luar (curang dikit, khusus buat riset), biar bisa observasi lebih jauh; apa level/round bakal naik kalau kita survive lebih lama & bunuh lebih banyak musuh.

```

// step4_cheat.js
for (let f = 0; f < 20000; f++) {
  x.t(4, 16);
  const s = z();
  if (s[o.al] === 0 && s[o.ph] === 1) { s[o.al] = 1; s[o.ph] = 0; } // cheat
  if (s[o.ph] !== 0) {
    console.log('END', s[o.sc], s[o.lv], s[o.ro]);
    console.log(x.v(), x.l());
    break;
  }
}
}

```

```
EVIDENCE
> node step4 cheat.js
END frame 6186 | score= 1775 | lv= 1 | round(ro)= 0
v() = 0 | l() = 0 <-- masih 0, belum menang
PS C:\Users\LENOVO\Downloads\maniac\evidence>
```

Walau survive penuh 90 detik + bunuh banyak musuh, lv & ro TETEP di nilai awal, dan v()/l() masih 0 belum menang

Ini janggal. Kalau memang mekanismenya 'bunuh musuh sampai level naik', harusnya ada sesuatu yang berubah. Tapi lv dan ro gak gerak sama sekali walau udah cheat survive penuh durasi. Kesimpulan sementara saya: kayaknya menang lewat main biasa (bahkan dibantu cheat survive) emang gak didesain buat kejadian. Dari sini saya mutusin harus baca bytecode WASM-nya langsung, gak bisa cuma coba-coba dari luar.

5. Disassemble WASM

5.1 wabt - gagal

Saya coba pertama pake package wabt (official WebAssembly Binary Toolkit).

```
npm install wabt
```

```
// dis_wabt.js
const wabt = await wabtInit();
const mod = wabt.readWasm(fs.readFileSync('maniac.wasm'),
{readDebugNames:false});
```

```
EVIDENCE
> node dis_wabt.js
GAGAL: readwasm failed:
0000c6b: error: unable to read u32 leb128: string length
PS C:\Users\LENOVO\Downloads\maniac\evidence>
```

wabt gagal parsing dengan error 'unable to read u32 leb128: string length' di offset 0xc6b

Saya sempet curiga ini trik anti-disassembly (byte-byte sengaja dirusak). Tapi setelah saya cek manual pakai parser section WASM sendiri (baca header + tiap section id/size satu-satu), file-nya ternyata valid & lengkap sampai byte terakhir. Jadi errornya bukan file korup, lebih ke ada encoding tertentu di bytecode yang gak disukai wabt secara strict. Kemungkinan LEB128 non-canonical yang tetap valid buat V8 tapi ditolak wabt. Saya putuskan pindah tool.

5.2 binaryen - berhasil

```
npm install binaryen
```

```
// dis_binaryen.mjs
import binaryen from 'binaryen';
const mod = binaryen.readBinary(fs.readFileSync('maniac.wasm'));
fs.writeFileSync('maniac.wat', mod.emitText());
```

```
EVIDENCE
> node dis_binaryen.mjs
sukses, cek maniac.wat, jumlah baris: 2393
PS C:\Users\LENOVO\Downloads\maniac\evidence> _
```

binaryen berhasil, menghasilkan file .wat sekitar 2400 baris

Dari file .wat ini saya cari daftar export dulu buat orientasi.

```
EVIDENCE
> Select-String -Path maniac.wat -Pattern export
maniac.wat:18: (export "m" (memory $0))
maniac.wat:19: (export "i" (func $20))
maniac.wat:20: (export "t" (func $21))
maniac.wat:21: (export "s" (func $22))
maniac.wat:22: (export "v" (func $23))
maniac.wat:23: (export "l" (func $24))
PS C:\Users\LENOVO\Downloads\maniac\evidence>
```

6 fungsi export: m (memory), i=\$20 (init), t=\$21 (tick), s=\$22, v=\$23 (cek menang), l=\$24 (panjang flag)

6. Baca Fungsi v() (func \$23) - Gerbang Kemenangan

Fungsi \$23 ini yang paling krusial karena itu yang dipanggil JS buat cek menang. Isinya satu rantai AND panjang berisi perbandingan i32.eq ke berbagai index state. Ini potongan awalnya:

```
EVIDENCE
(i32.and
  (i32.and
    (i32.and
      (i32.and
        (i32.and
          (i32.eq
            (call $0
              (i32.const 3)
            )
          )
        )
      )
    )
  )
  (i32.const 54921)
)
(i32.eq
  (call $0
    (i32.const 21)
  )
)
(i32.const 54921)
)
(i32.eq
  (call $0
    (i32.const 4)
  )
)
(call $0
  (i32.const 15)
)
)
)
(i32.eq
  (call $0
    (i32.const 11)
  )
)
(call $0
  (i32.const 16)
)
)
)
(i32.eq
  (call $0
    (i32.const 12)
  )
)
)
(i32.const 511)
)
```

Isi awal fungsi v() - rantai kondisi AND: idx3==54921, idx21==54921, idx4==idx15, idx11==idx16, idx12==511, dst

Setelah saya baca semua, syarat lengkapnya kira-kira begini (disederhanain dari bytecode):

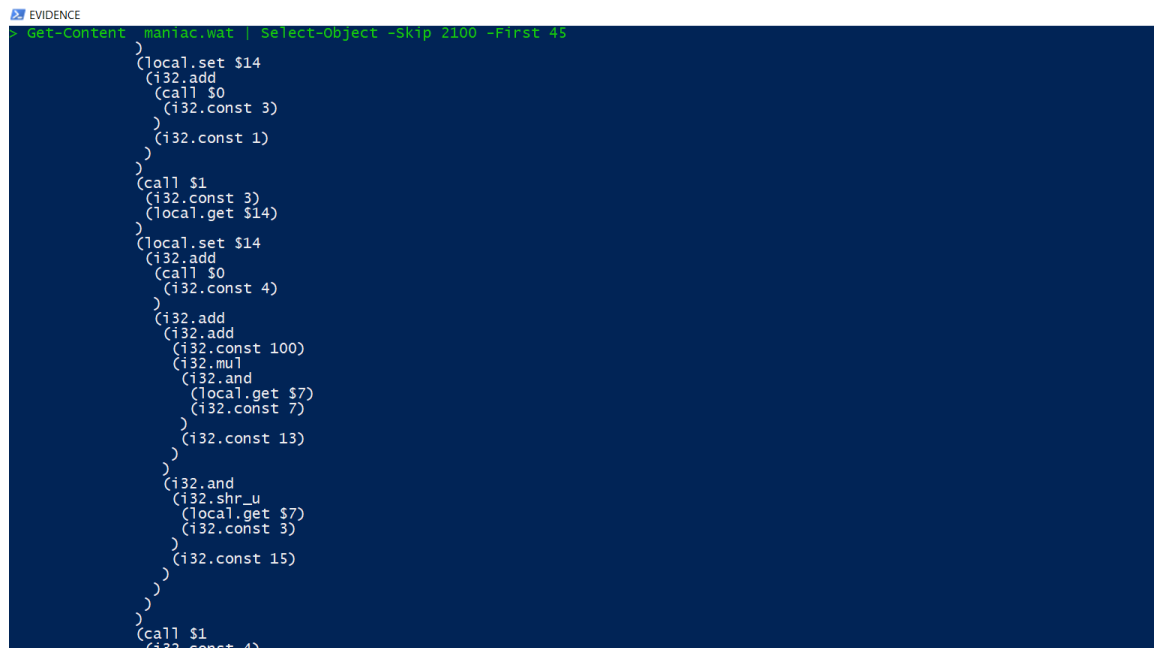
```
win = (idx3 == 54921)      // 'cl' - kill counter?
      && (idx21 == 54921)   // nilai konstan yang udah ada dari awal init
      && (idx4 == idx15)     // 'sc' (score) harus == 'es' (target score)
      && (idx11 == idx16)    // 'au' harus == 'ea'
      && (idx12 == 511)      // 'ms'
      && (idx10 == 0)        // 'dy' - kayaknya death count, harus 0
      && (idx5 == 1)        // 'al' - alive
      && (idx6 == 0)        // 'ph' - masih 'playing'
      && (idx9 == 9)        // 'ro' - round
      && (idx7 > 0)         // 'tm' - waktu tersisa
      && (hash(...) == idx17) // integrity check

if (win) { ph = 2; call decrypt_flag(); return pointer_ke_flag; }
```

Field terakhir (hash check) itu awalnya saya kira semacam snapshot integrity yang ketat, tapi setelah saya trace lebih jauh, ternyata idx17 di-refresh ulang tiap akhir tick() pake formula hash yang sama - jadi kondisi itu praktisnya selalu true (bukan pembandingan ke snapshot lama). Yang beneran jadi tantangan adalah 10 kondisi equality di atas.

7. Trace idx3 (Kill Counter) - Ketemu Kontradiksi Besar

Syarat paling mencurigakan buat saya adalah $idx3 == 54921$. Angka segini gede banget buat sebuah 'counter' biasa. Saya cari di mana aja idx3 ini ditulis di seluruh fungsi tick(), dan ketemu di collision-handler (bagian kode yang jalan pas bullet nabrak musuh):



```
> Get-Content maniac.wat | Select-Object -Skip 2100 -First 45
    (local.set $14
    (i32.add
    (call $0
    (i32.const 3)
    )
    (i32.const 1)
    )
    )
    (call $1
    (i32.const 3)
    (local.get $14)
    )
    (local.set $14
    (i32.add
    (call $0
    (i32.const 4)
    )
    (i32.add
    (i32.add
    (i32.const 100)
    (i32.mul
    (i32.and
    (local.get $7)
    (i32.const 7)
    )
    (i32.const 13)
    )
    (i32.and
    (i32.shr_u
    (local.get $7)
    (i32.const 3)
    )
    (i32.const 15)
    )
    )
    )
    (call $1
    (i32.const 4)
```

*$idx3 = idx3 + 1$, dijalankan tiap kali musuh berhasil dibunuh. Juga kelihatan formula score: $100 + (type \& 7) * 13 + ((type >> 3) \& 15)$*

Ini konfirmasi: idx3 murni kill counter, naik persis +1 per kill, gak ada jalur lain yang nulis ke situ selain di init (=0). Saya verifikasi juga sekalian hitung-hitungan matematisnya:

```
// step8_impossible.js
console.log('Target cl harus:', 54921);
console.log('Ammo maksimal (gak regen):', s[o.am]); // 96
console.log('Target sc harus == es:', s[o.es]);
console.log('Max score realistis (96 kill x ~200):', s[o.am] * 200);
```

```

EVIDENCE
> node step8_impossible.js
=== Syarat menang (dari analisis func v()) vs kenyataan ===
Target c1 (idx3, kill counter) harus : 54921
Ammo maksimal yang tersedia (idx2) : 96 <-- gak regen, jadi max kill ~ 96
Target sc (idx4, score) harus sama dgn es (idx15) : 8414013
Max score realistis ( 96 kill x ~200 poin) : 19200
KESIMPULAN: target-target ini MATEMATIS MUSTAHIL dicapai lewat main normal.
PS C:\Users\LENOVO\Downloads\maniac\evidence>

```

Target kill counter 54921 vs ammo maksimal cuma 96. Target score 8.414.013 vs max realistis ~19.200

Di titik ini semuanya jadi jelas: target-target yang diminta fungsi `v()` itu matematis mustahil dicapai lewat main normal berapa kali pun dicoba. Ammo cuma 96 (gak regen), tapi butuh kill counter 54921. Score max realistis ~19 ribu. Ini yang bikin saya sadar clue "cheesy" & "cuma bisa sekali coba" di deskripsi tadi maksudnya: game ini memang didesain unwinnable lewat UI. Jalan keluarnya bukan main lebih jago, tapi manipulasi state memory langsung + minta WASM-nya sendiri yang "decrypt" flag.

8. Cari Fungsi Decrypt Flag

Di dalam fungsi `v()`, pas kondisi menang true, dia manggil satu fungsi lain (`func $19` di build saya) sebelum return pointer flag. Isinya kurang lebih gini setelah saya baca & sederhanain:

```

len = mem8[8192]           // panjang ciphertext = 31 byte
k3 = h6(sd, cl, es)
k4 = h6(ea, ms, ro)
k5 = h6(k3, k4, tk)
k6 = h6(k4, k5, lv)

prev = -1515870811         // IV
for i in 0..len:

```

```

c      = mem8[8193 + i]    // byte ciphertext
keystream = f18(k3, k4, k5, k6, i) & 0xff
plain   = c XOR prev XOR keystream
mem8[12288 + i] = plain
prev = c                  // chaining pakai ciphertext

```

Pola ini stream cipher custom - keystream dari fungsi hash (mirip gaya MurmurHash3 finalizer: kombinasi add/xor/rotl/imul/shift), plus chaining ala CFB pakai byte ciphertext sebelumnya. Ciphertext 31 byte-nya sendiri saya temuin persis di 31-byte terakhir file maniac.wasm.

Field-field yang jadi kunci decrypt (sd, cl, es, ea, ms, ro, tk) itu sama persis dengan field-field yang dicek di v(). Jadi begitu tau nilai targetnya dari v(), otomatis tau juga kunci buat decrypt-nya; kecuali satu: lv, yang gak ada constraint eksplisit di v().

9. Solve Final - Injeksi State + Brute Force

Daripada re-implementasi seluruh algoritma hash & keystream itu manual di JS (rawan salah kecil yang bikin hasil decrypt jadi garbage - dan ini beneran sempet kejadian di percobaan awal saya), saya pilih pendekatan yang lebih aman: suntik langsung nilai-nilai target yang udah pasti bener (dari hasil baca v()) ke memory WASM, terus panggil v() dan l() yang ASLI - biar WASM sendiri yang decrypt flag-nya.

```

async function attempt(lv) {
  // instance BARU tiap percobaan - v() punya 'latch', sekali sukses hasil ke-cache
  const inst = await WebAssembly.instantiate(wasmBuf, {});
  const x = inst.instance.exports;
  x.i(0x13579bdf, 90000, 0);
  const s = new Int32Array(x.m.buffer);
  const sd = s[14], es = s[15], ea = s[16];

  s[3]=54921; s[4]=es; s[11]=ea; s[12]=511; s[10]=0;
  s[5]=1; s[6]=0; s[9]=9; s[7]=1; s[18]=lv;

  // rekonstruksi ulang hash idx17 (tk) biar cek integrity di v() lolos
  s[17] = h6(h6(h6(h6(sd,54921,es), ea,511), es,ea), 9, lv);

  const ptr = x.v();
  const len = x.l();

```

```

return len ? Buffer.from(x.m.buffer, ptr, len).toString('latin1') : null;
}

for (let lv = 0; lv < 20; lv++) {
  const flag = await attempt(lv);
  if (flag && flag.startsWith('TryHards')) { console.log('FOUND!', lv, flag); break; }
}
}
}

```

lv gak dicek eksplisit di v(), jadi saya brute-force dari 0. Karena v() nge-latch hasil (sekali sukses, panggilan berikutnya cuma return cache lama), saya harus bikin instance WASM baru tiap percobaan biar tiap lv beneran dievaluasi ulang dari nol.

```

# EVIDENCE
# node step9_10_solve.js
Brute forcing nilai lv (satu-satunya unknown)...

lv = 0 -> "tSEj"iU\u001f\u001fâ6"n\u00\u0003Az#i.Zazn\u00004\u0000eè*M"
lv = 1 -> "b\u0001\u001e\u00012\u00Dx\u00E0wâtp\u001q\u00015\u00002\u006A\u00181P"
lv = 2 -> "A\u0001d\u00E0a\u001f\u00D\u0000e\u00005P-76\u00\u00005è\u0018ek\u0013o"
lv = 3 -> "uGpD#u\u0000\u00013.%#G0S9\u001E\u00000bK\u00"
lv = 4 -> "u0003#ZU:ç 000\u00017\u00wji\u001000\u00'x\u00'S<?A"
lv = 5 -> "â\u00E0\u0010p\u00\u0015-BS\u00Dw1m\u000186\u00\u001E(±)"
lv = 6 -> "u001a1u\u00\u0000511\u000018\u0012\u001E\u0010\u0001q\u001f\u0018\u00"
lv = 7 -> "e\u001dEèç\u00C2\u001f\u000000E\u0018\u00018b0c\u00K\u00v\u0015e">"
lv = 8 -> "u0017p\u001f01\u000048741e\u00F6 w\u00n tw\u0080 000L"
lv = 9 -> "â\u00\u00003\u0000\u001d0n\u00\u00b0p\u007t/\u00\u0013.-baAs"
lv = 10 -> "TryHards{m4n!ac_sh00o0o0o0o0tzz}"

=== FLAG KETEMU! lv = 10 ===
TryHards{m4n!ac_sh00o0o0o0o0tzz}
PS C:\Users\LENOVO\Downloads>maniac\evidence>

```

Brute force lv dari 0 - semua hasil garbage/random sampai lv=10, keluar
TryHards{m4n!ac_sh0o0o0o0o0tzz}

10. Flag

TryHards{m4n!ac_sh0o0o0o0o0tzz}

11. Ringkasan Alur

4. Baca index.html - WASM standalone (no import), seed & durasi fixed → fully deterministic.

5. Load & eksperimen langsung dari Node.js (lebih cepat dari browser) - ketemu 6 export, dan ammo terbatas (96, gak regen).
6. Coba main manual + cheat survive - level/round gak pernah naik walau survive penuh durasi. Sinyal ada yang janggal.
7. wabt gagal disassemble (encoding aneh) → pindah ke binaryen, berhasil dapet file .wat.
8. Baca fungsi v() - nemu 10+ kondisi equality yang harus dipenuhi buat menang.
9. Trace field idx3 (kill counter) - ketahuan cuma +1 per kill, target 54921 gak mungkin dicapai dengan ammo cuma 96.
10. Kesimpulan: game didesain unwinnable lewat UI. Solve sebenarnya lewat manipulasi memory langsung.
11. Baca fungsi decrypt (func \$19) - stream cipher custom, ciphertext ada di 31 byte terakhir file wasm.
12. Injeksi nilai-nilai target yang sudah pasti benar ke memory WASM, brute force satu-satunya unknown (lv), pakai instance baru tiap percobaan.
13. lv = 10 menghasilkan flag valid: TryHards{m4n!ac_sh0o0o0o0o0tzz}